BANANA PROBLEM 4

SUBJECT: UE24CS242B - Operating Systems

NAME: K ASHRITHA REDDY

SRN: PES2UG24AM071

SECTION: 4A (AIML)

Github repository link:
https://github.com/LogicNinja/PES2UG24AM071-pes-vcs

Phase 1: Object Storage Foundation

Screenshot 1A: Output of ./test_objects showing all tests passing.

```
seed@seedvm2026:~/PES2UG24AM071-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db
5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0
221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B: find .pes/objects -type f showing the sharded directory structure.

```
seed@seedvm2026:~/PES2UG24AM071-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4
f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e56
6fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aa
fc12
seed@seedvm2026:~/PES2UG24AM071-pes-vcs$
```

Phase 2: Tree Objects

Screenshot 2A: Output of ./test_tree showing all tests passing.

```
seed@seedvm2026:~/PES2UG24AM071-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B: Pick a tree object from find .pes/objects -type f and run xxd .pes/objects/XX/YYY... | head -20 to show the raw binary format.

```
seed@seedvm2026:~/PES2UG24AM071-pes-vcs$ find .pes/objects -type f
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
seed@seedvm2026:~/PES2UG24AM071-pes-vcs$ xxd .pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b05
7b53b14b4b9b939dd74decc5321 | head -20
00000000: 7472 6565 2030 00                          tree 0.
```

Phase 3: The Index (Staging Area)

📸 Screenshot 3A: Run ./pes init, ./pes add file1.txt file2.txt, ./pes status — show the output.

```
seed@seedvm2026:~/PES2UG24AM071-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
seed@seedvm2026:~/PES2UG24AM071-pes-vcs$ echo "hello" > file1.txt
seed@seedvm2026:~/PES2UG24AM071-pes-vcs$ echo "world" > file2.txt
seed@seedvm2026:~/PES2UG24AM071-pes-vcs$ ./pes add file1.txt
seed@seedvm2026:~/PES2UG24AM071-pes-vcs$ ./pes add file2.txt
seed@seedvm2026:~/PES2UG24AM071-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_tree.c
  untracked:  commit.h
  untracked:  tree.c
  untracked:  README.md
  untracked:  test_objects.c
  untracked:  pes.c
  untracked:  index.c
  untracked:  test_sequence.sh
  untracked:  file.txt
  untracked:  object.c
  untracked:  .gitignore
  untracked:  pes.h
  untracked:  Makefile
  untracked:  commit.c
  untracked:  index.h
  untracked:  tree.h
```

📸 Screenshot 3B: cat .pes/index showing the text-format index with your entries.

```
seed@seedvm2026:~/PES2UG24AM071-pes-vcs$ cat .pes/index
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 17
76451479 6 file1.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 17
76451485 6 file2.txt
```

Phase 4: Commits and History

Screenshot 4A: Output of ./pes log showing three commits with hashes, authors, timestamps, and messages.

```
seed@seedvm2026:~/PES2UG24AM071-pes-vcs$ echo "third change" >> file1.txt
seed@seedvm2026:~/PES2UG24AM071-pes-vcs$ ./pes add file1.txt
seed@seedvm2026:~/PES2UG24AM071-pes-vcs$ ./pes commit -m "Third commit"
Committed: f6b1b2704a1b... Third commit
seed@seedvm2026:~/PES2UG24AM071-pes-vcs$ ./pes log
commit f6b1b2704a1b739356209dc16a6331ebf82c221d2c6fe5dbee6e71795978fc85
Author: PES User <pes@localhost>
Date:   1776452427

    Third commit


commit e4630b884c9aeae2bf2f1677c26a523a99fe9d203c5c06c5e89779260df9fa7b
Author: PES User <pes@localhost>
Date:   1776452319

    Updated file


commit 52c08c90ce7b33264ff9bc542a6129ffda0224088c36d0f065fcbce6d4c7db5a
Author: PES User <pes@localhost>
Date:   1776452297

    Initial commit
ial commiQ
```

Screenshot 4B: find .pes -type f | sort showing object store growth after three commits.

```
seed@seedvm2026:~/PES2UG24AM071-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9
ebb4
.pes/objects/2d/03278b2daa250ae541099d4bd388f59a20d6d8c7739006a4258c316622
b2f6
.pes/objects/3a/3e22c686474097b70bc8a8e46e13e1046fbcdc09fac7796143ecbfb422
f05c
.pes/objects/4e/f382b6f9f0130c7521006ce21bdb7c9905dfe0ef0dcb6ede208de60d9e
33d7
.pes/objects/52/c08c90ce7b33264ff9bc542a6129ffda0224088c36d0f065fcbce6d4c7
db5a
.pes/objects/93/a8ca6711758248ec12b340445a51473facc6ec3872d0238a1b8a72307f
6fcb
.pes/objects/e4/630b884c9aeae2bf2f1677c26a523a99fe9d203c5c06c5e89779260df9
fa7b
.pes/objects/f6/b1b2704a1b739356209dc16a6331ebf82c221d2c6fe5dbee6e71795978
fc85
.pes/objects/fe/76325aa5521b207ebe01e12fd8e9e3abf030cacd5398e3744a3a56a81a
d1bd
.pes/refs/heads/main
```

Screenshot 4C: cat .pes/refs/heads/main and cat .pes/HEAD showing the reference chain.

```
seed@seedvm2026:~/PES2UG24AM071-pes-vcs$ cat .pes/refs/heads/main
f6b1b2704a1b739356209dc16a6331ebf82c221d2c6fe5dbee6e71795978fc85
seed@seedvm2026:~/PES2UG24AM071-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Phase 5 & 6: Analysis-Only Questions

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

To implement pes checkout <branch>, the HEAD file must be updated to point to the selected branch. Then, the commit hash from .pes/refs/heads/<branch> is used to retrieve the corresponding tree.

The index is updated to match this tree, and the working directory is modified accordingly: files are created, updated, or deleted to reflect the target commit.

This operation is complex because it must keep HEAD, index, and working directory consistent, while safely handling file differences and avoiding overwriting user changes.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

First, compare each tracked file in the working directory with the index using hashes. If they differ, the file has uncommitted changes.

Then, compare the index with the target branch’s tree. If a file differs here, it will be modified during checkout.

A conflict occurs if a file is both modified in the working directory and different in the target branch. In such cases, checkout must be refused.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

In detached HEAD, HEAD points directly to a commit instead of a branch.

New commits can still be made, but no branch points to them, so they may become unreachable and get deleted later.

To recover, the user can create a new branch pointing to that commit or use the commit hash before garbage collection removes it.

Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or

transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Start from all branch heads and traverse commits, trees, and blobs using DFS or BFS, marking all reachable objects.

Store visited hashes in a hash set for efficiency. After traversal, delete any object not in this set.

For ~100,000 commits and 50 branches, traversal will visit roughly 100,000 commits plus related trees and blobs, since branches often share history.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

If GC runs during a commit, it may delete newly created objects that are not yet referenced by a branch.

This causes corruption when the commit later tries to reference those objects.

Git prevents this using atomic writes, temporary files, locking, and delaying deletion of recent objects.